

Simulador do Modelo OSI

1. Visão geral

O simulador representa uma comunicação em rede segundo o modelo OSI. Os computadores (hosts) participam das sete camadas, enquanto os roteadores processam somente as camadas Física, Enlace e Rede. A execução da simulação é transformada em uma sequência de eventos estruturados, posteriormente consumida pela interface gráfica. A topologia da rede é carregada a partir de um arquivo JSON.

O projeto foi organizado de forma modular para separar a representação das PDUs, o comportamento das camadas, os dispositivos, a rede, a coordenação da simulação, os eventos e a interface gráfica.

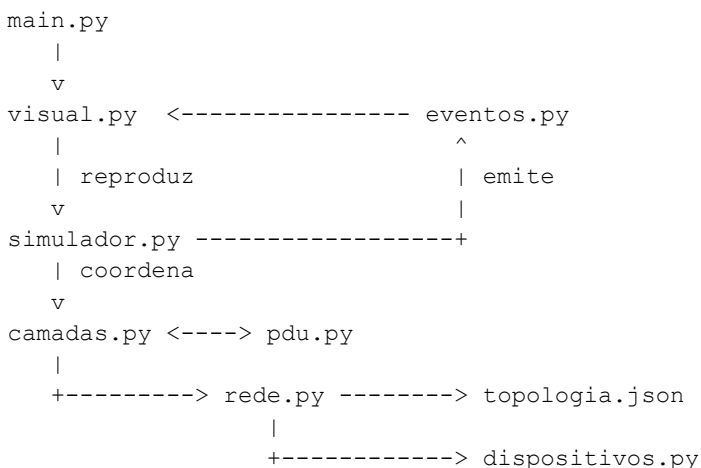
2. Separação entre as camadas

A lógica principal do modelo OSI encontra-se na classe `PilhaOSI`, em `camadas.py`. O fluxo de envio progride na ordem $L7 \rightarrow L6 \rightarrow L5 \rightarrow L4 \rightarrow L3 \rightarrow L2 \rightarrow L1$. No destino, o fluxo é reconstruído no sentido inverso.

Entre saltos, o quadro da camada 2 é removido e um novo quadro é criado para o enlace seguinte. Dessa forma, os endereços físicos e o identificador do quadro podem mudar a cada salto, enquanto os endereços lógicos de origem e destino permanecem constantes de ponta a ponta.

A restrição dos roteadores é representada estruturalmente em `dispositivos.py`: Roteador herda somente os dados genéricos de `Dispositivo` e não possui estado ou métodos relativos a sessão, portas, aplicação ou apresentação. Durante o encaminhamento, `camadas.py` executa no roteador somente eventos das camadas 1, 2 e 3. A porta permanece como informação da PDU de transporte, mas não é consultada para decidir o próximo salto.

3. Diagrama de módulos e fluxo



Fluxo de dados:

Aplicação → Apresentação → Sessão → Transporte → Rede
→ Enlace → Física → [enlace/roteadores] → Física → Enlace
→ Rede → Transporte → Sessão → Apresentação → Aplicação

4. Estrutura da unidade de dados

Campo	Tipo	Função
dados	bytes	Conteúdo útil atualmente transportado.
destino_nome / origem_nome	str	Identificação dos hosts da simulação.
processo_origem / processo_destino	int	Portas dos processos.
codificacao	str	Codificação textual; nesta versão, UTF-8.
cifrado	bool	Indica se o conteúdo está cifrado.
sessao_id	str	Identificador da sessão.
portas	tuple	Par porta de origem/destino.
logicos	tuple	Par de IPs de origem/destino, mantido ponta a ponta.
fisicos	tuple	Par de MACs do salto corrente.
quadro_id	str	Identificador Qn do quadro corrente.
verificacao	int	Soma dos octetos módulo 256 usada para detectar alteração.
segmento_indice / segmento_total	int	Controle da segmentação e remontagem.
unidade	str	mensagem, segmento, pacote ou quadro.
cabecalhos	list	Lista (nome, tamanho) dos cabeçalhos presentes.
tem_trailer_l2	bool	Indica presença do finalizador de L2.

5. Interface dos módulos

Módulo / elemento	Recebe	Devolve / efeito
main.py	Nenhum parâmetro de usuário	Cria Tk, instancia GUI e inicia mainloop.
GUI	root Tk	Constrói a interface e reproduz eventos.
SimuladorOSI.executar	origem, destino, mensagem, portas	Lista de eventos gerados.
SimuladorOSI.caso	n de 1 a 7	Configura e executa o cenário escolhido.
SimuladorOSI.eficiencia	estado da execução	η e sobrecarga.
BarramentoEventos.emitir	dispositivo, camada, ação, descrição, PDU	Evento numerado e armazenado.
PilhaOSI.enviar	origem, destino, mensagem, portas	Produz a sequência de envio e chama o transporte de cada segmento.
PilhaOSI._transportar_segmento	PDU de segmento	Encapsula L3/L2, transmite pelos saltos e entrega ao destino.
PilhaOSI.finalizar_recebimento	PDU remontada	Remove L4/L5, decifra e entrega a L7.
Rede.menor_caminho	origem, destino	Caminho e custo por Dijkstra.
Rede.par_interfaces	dispositivo atual e próximo	Interfaces usadas no salto.
PDU.tamanho_atual	estado da PDU	Tamanho total corrente em bytes.
PDU.blocos_visuais	estado da PDU	Lista de blocos usada pela interface.

6. Parâmetros configuráveis

Parâmetro	Valor desta versão	Faixa / observação	Local
Cabeçalho L5	4 B	Constante não negativa	pdu.py: TAM_C5
Cabeçalho L4	8 B	Constante não negativa	pdu.py: TAM_C4
Cabeçalho L3	20 B	Constante não negativa	pdu.py: TAM_C3
Cabeçalho L2	14 B	Constante não negativa	pdu.py: TAM_C2
Finalizador L2	4 B	Constante não negativa	pdu.py: TAM_F2
Limite de segmento	40 B	Inteiro positivo	camadas.py: LIMITE_SEGMENTO
Topologia	topologia.json	JSON compatível	rede.py / src/
Velocidade lenta	1200 ms/evento	Valor positivo	visual.py
Velocidade normal	600 ms/evento	Valor positivo; padrão	visual.py
Velocidade rápida	180 ms/evento	Valor positivo	visual.py
Chave da cifra XOR	0x5A	0 a 255	camadas.py: xor_bytes

7. Funcionamento interno

Na origem, L7 cria a mensagem. L6 converte o texto para octetos UTF-8 e aplica uma cifra XOR didática. L5 cria o identificador de sessão e acrescenta o cabeçalho de sessão. L4 divide os dados em fatias de até 40 bytes e registra portas e numeração dos segmentos.

L3 define os IPs de origem e destino e consulta Rede para obter o caminho. Em cada salto, L2 seleciona as interfaces do dispositivo atual e do próximo dispositivo, cria um novo identificador de quadro, define os MACs, acrescenta cabeçalho/finalizador e calcula a verificação. L1 registra a transmissão em bits.

Na recepção de cada salto, a verificação é recalculada. Se houver diferença, L2 registra DESCARTA e o processamento daquele quadro termina. Se estiver correta, L2 remove cabeçalho e finalizador. Se o receptor do salto for roteador, somente L3 decide o próximo salto; um novo quadro é então construído.

No host final, L3 entrega o segmento a L4. Os segmentos são armazenados em buffer por fluxo/sessão. Quando todos chegam, são ordenados pelo índice e concatenados. L4 remove seu cabeçalho, L5 encerra a sessão, L6 aplica novamente XOR para decifrar e converte UTF-8 para texto, e L7 registra a entrega ao processo.

8. Convenções de simulação

Os tamanhos de cabeçalhos são os definidos em pdu.py. O limite de dados por segmento é 40 bytes. O cabeçalho de sessão é preservado como parte do estado criado antes da segmentação e copiado para os segmentos. A numeração de sessão e quadro utiliza contadores crescentes no processo.

A verificação da camada 2 é a soma dos octetos módulo 256. Trata-se de uma convenção didática adotada pelo simulador e não de um CRC de produção.

O algoritmo de menor caminho é Dijkstra. Quando existem custos iguais, o comportamento de desempate decorre da ordem de inserção e da ordenação natural da fila de prioridade usada pela implementação. Um quadro detectado como corrompido é descartado imediatamente e não sobe para L3.

9. Eventos e interface

BarramentoEventos mantém uma sequência numerada. Cada Evento registra passo, dispositivo, camada, ação, descrição, tamanho, unidade, IPs, MACs, quadro, caminho e blocos da PDU.

A interface não chama as camadas para descobrir como desenhar um passo; ela reproduz a lista de eventos já produzida pelo simulador. O log usa o formato “passo | dispositivo | camada | ação | descrição | tamanho B”. O mapa utiliza o caminho presente no evento; as pilhas usam dispositivo e camada; o desenho da PDU usa pdu_blocos; e os painéis de endereçamento usam lógicos e físicos.

10. Eficiência e sobrecarga

SimuladorOSI acumula tamanho_util com os bytes da mensagem original e total_transmitido com o tamanho de cada quadro efetivamente transmitido. A eficiência é calculada como $\eta = \text{tamanho_util} / \text{total_transmitido}$ e a sobrecarga como $1 - \eta$. A interface mostra os dois valores e, nos cenários C1 e C2, apresenta uma comparação.

11. Cenários C1-C7

Caso	Configuração interna
C1	H1 → H2, entrega direta.
C2	H1 → H4, entrega indireta.
C3	Fluxos H1 → H4 e H2 → H4, com portas de origem distintas.
C4	Desativa R1–R4 antes da comunicação H1 → H4.
C5	Cria destino lógico 10.0.9.10 e registra ausência de rota.
C6	Configura alteração de um bit no salto R4 → R3.
C7	Envia mensagem longa, excedendo 40 bytes e produzindo vários segmentos.

12. Dependências e ambiente

A aplicação é escrita em Python 3 e utiliza módulos da biblioteca padrão para sua execução, incluindo tkinter/ttk, json, heapq, dataclasses, copy e itertools. O executável Windows é gerado com PyInstaller pelo script build_exe.bat.

O script de build instala PyInstaller e executa o empacotamento em modo one-file e windowed. Na versão atual do build, topologia.json é incluído no empacotamento.

13. Limitações e pontos de validação antes da entrega

A versão final deve ser testada em Windows sem ambiente Python, confirmando que o executável abre por duplo clique.

O guia da disciplina informa que a topologia deve poder ser substituída sem gerar novo executável. Como o build one-file atual incorpora topologia.json, esse comportamento deve ser ajustado caso o professor teste a substituição do JSON externo ao lado do executável.

Também é necessário conferir a convenção de eficiência adotada pelo professor para o cenário de referência. A documentação do projeto deve registrar claramente a fórmula e a forma de contabilização efetivamente adotadas na versão entregue.

14. Manutenção

Para alterar a rede, edita-se `topologia.json` mantendo o esquema esperado. Para alterar tamanhos de cabeçalho, modificam-se as constantes em `pdu.py`. Para mudar o limite de segmentação, altera-se `PilhaOSI.LIMITE_SEGMENTO`. Para criar novos cenários, adiciona-se uma ramificação em `SimuladorOSI.caso` e uma opção correspondente na interface.

15. Estrutura dos arquivos do projeto

Arquivo	Responsabilidade
<code>main.py</code>	Ponto de entrada da aplicação gráfica.
<code>visual.py</code>	Interface: mapa, pilhas, PDU, IP/MAC, controles, log e OSI/TCP-IP.
<code>simulador.py</code>	Coordena cenários, buffers, métricas e execução.
<code>eventos.py</code>	Define Evento e BarramentoEventos.
<code>camadas.py</code>	Implementa o fluxo funcional das camadas.
<code>pdu.py</code>	Representa a PDU e seus tamanhos/cabeçalhos.
<code>rede.py</code>	Carrega a topologia, controla enlaces e calcula caminhos.
<code>dispositivos.py</code>	Representa hosts e roteadores.
<code>topologia.json</code>	Configuração externa da rede.
<code>build_exe.bat</code>	Automatiza a geração do executável Windows.